AZURE DATA FACTORY · GUIDE N° 2 · PAS À PAS · NIVEAU AVANCÉ

Un pipeline qui audite chaque facture et décide seul de sa qualité

Un manifeste dicte les règles du lot. Le pipeline l'attend, compte les fichiers jusqu'à ce que tout soit arrivé, recalcule le prix juste de chaque session avec la grille tarifaire, classe les écarts, puis ouvre ou ferme lui-même une porte qualité avant d'archiver le lot à l'octet près et de vider la boîte de réception.

Validation · Recherche · Jusque (Until) · Attendre · Basculer (Switch) · Supprimer · Copie binaire · Source JSON · Recherche de data flow · Fenêtre · Sélectionner · Paramètre de data flow · Nouvelle branche

Par **Abdoul Hamid Diallo** · Microsoft Certified Fabric Data Engineer, Fabric Analytics Engineer et Azure AI Engineer

60 → 57 + 3
sessions lues, auditées, rejetées

11,05 €
de surfacturation détectés sur 5 sessions

1 porte
qualité qui accepte ou refuse le lot

0 ressource
Azure nouvelle : tout est réutilisé

Rappel : on reste dans le même environnement. Ce guide prolonge le guide n° 1 et réutilise **exactement** ses ressources : le groupe de ressources `rg-adf-charge`, la fabrique `adf-charge-XXXX`, le compte de stockage `stchargeXXXX`, le service lié `LS_ADLS_Charge` (identité managée) et le dataset `DS_ADLS_Csv`. On n'ajoute qu'un conteneur et deux datasets. Vous n'avez pas fait le guide n° 1 ? Faites d'abord ses étapes 1 et 2 (environ la moitié du guide n° 1), puis revenez ici.

Le scénario

Les trois stations de recharge de Metz, Nancy et Thionville envoient leurs sessions du 14 octobre. Le service financier soupçonne des erreurs de facturation : certaines sessions semblent payées plus cher que la grille tarifaire, d'autres moins.

La grille est simple mais piégeuse : un prix au kWh en **heures pleines**, un autre en **heures creuses**, des plages horaires propres à chaque station, et des frais fixes par session. Il faut recalculer le prix juste de chaque session, mesurer l'écart avec ce qui a été payé, et livrer un rapport par station.

Le lot ne doit être accepté que si sa qualité est suffisante. Sinon, rien ne doit être archivé ni effacé.

Le parcours

1. Préparer le conteneur guide2 et y déposer les fichiers
2. Créer deux datasets génériques de plus
3. Le data flow : typer et contrôler les sessions
4. Le data flow : appliquer la grille et classer les écarts
5. Le pipeline : attendre, vérifier, auditer
6. Le pipeline : la porte qualité, l'archive et le ménage
7. Publier, exécuter, vérifier, puis éprouver la porte qualité
8. Tout fermer pour ne plus rien consommer

Les fichiers joints à ce PDF

Ouvrez le panneau **Pièces jointes** (le trombone) de votre lecteur PDF, puis enregistrez :

Fichier	Rôle
`run_manifest.json`	les règles du lot : nom, fichiers attendus, seuils
`tariffs.json`	la grille tarifaire des 3 stations
3 fichiers `sessions_*.csv`	60 sessions, dont 3 abîmées

Comment lire ce guide

Chaque valeur à saisir est dans un **bloc sombre**, à copier telle quelle. Les menus sont donnés en français, avec l'anglais entre parenthèses.

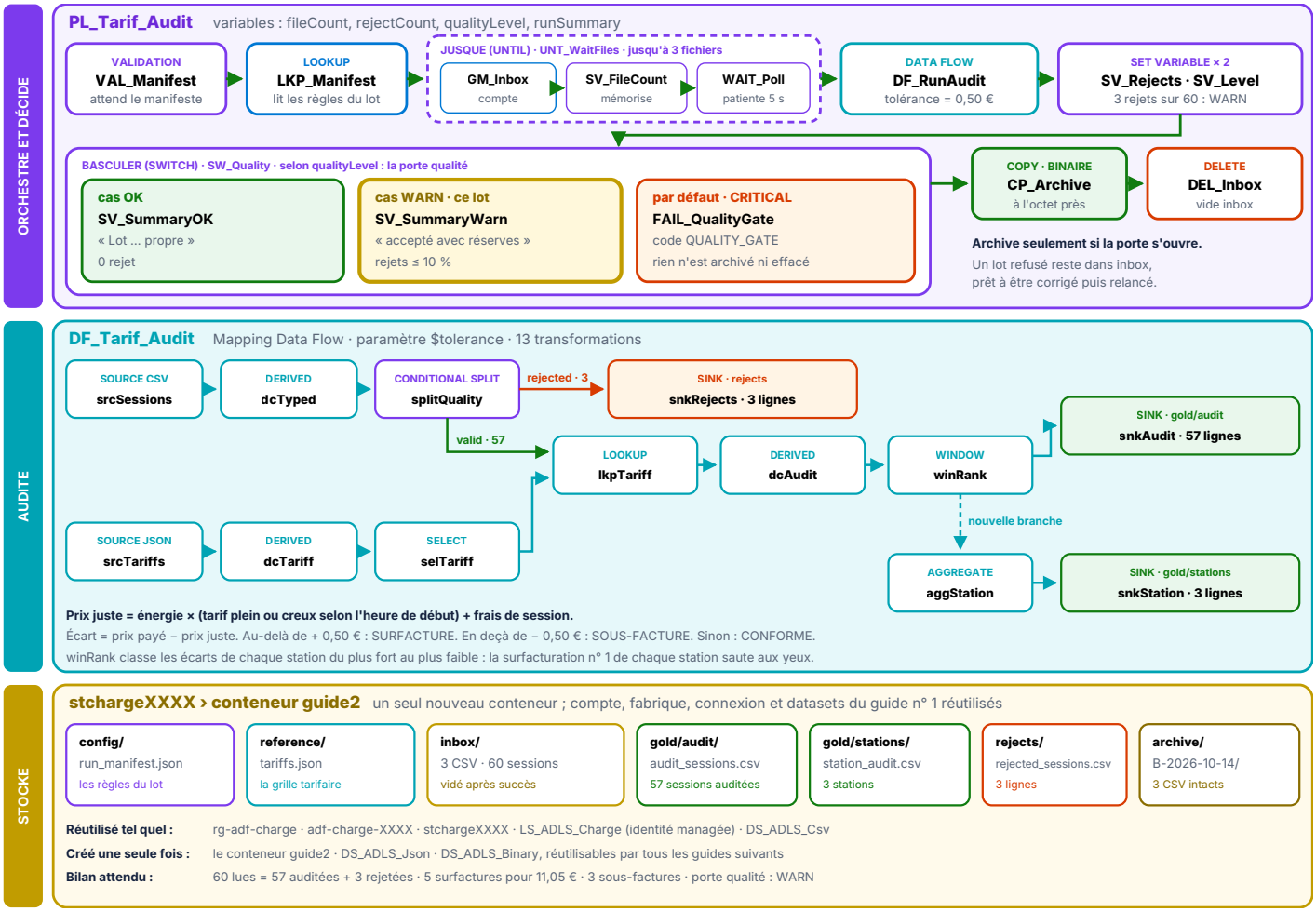
Dans le volet **Activités** en français : Switch s'appelle **Basculer**, Until **Jusque** (catégorie **Itération et conditions**), Lookup **Recherche**, Delete **Supprimer** ; Validation, Attendre et Échec sont dans **Général**.

Une valeur qui commence par `@` se colle via **Ajouter du contenu dynamique** (Add dynamic content).

Ce que ce pipeline a de plus que le guide n° 1. Il ne se contente plus de transformer : il **attend** son contrat (le manifeste), **patiente** jusqu'à ce que le lot soit complet, **confronte** chaque ligne à une référence, **juge** la qualité du lot avec des seuils pilotés de l'extérieur, et **n'efface jamais** ce qu'il n'a pas d'abord archivé.

La carte complète du pipeline

Trois couches, de haut en bas : **ce qui orchestre et décide**, **ce qui audite**, **où vivent les données**. Les nombres sur les flèches sont ceux que vous retrouverez à l'étape 7.



violet : contrôle du flux et décisions · bleu : lecture · turquoise : transformation Spark · vert : écriture, succès · orange : rejet, échec, suppression · or : stockage

Le manifeste commande
Le nom du lot, le nombre de fichiers attendus, le seuil de rejet et la tolérance d'audit vivent dans un fichier JSON. On change une règle sans republier le pipeline.

Deux sources, une jointure
Les sessions (CSV) et la grille (JSON) se rejoignent par station. Chaque session reçoit son prix juste, son écart et un verdict.

La porte qualité
OK, WARN ou CRITICAL. Les deux premiers acceptent le lot et l'archivent ; le troisième arrête tout avec un message clair.

Supprimer en dernier
La boîte de réception n'est vidée qu'après une archive réussie, et seulement si la porte s'est ouverte.

1 Préparer le conteneur guide2

Dans le portail, ouvrez **stchargeXXXX** › **Stockage des données** › **Conteneurs** › **+ Conteneur** :

```
guide2
```

Ouvrez **guide2** › **Charger**. Pour chaque dépôt, dépliez **Avancé** et remplissez **Charger dans le dossier** : le dossier se crée tout seul.

Charger dans le dossier	Fichiers
`config`	`run_manifest.json`
`reference`	`tariffs.json`
`inbox`	`sessions_metz_20261014.csv`, `sessions_nancy_20261014.csv`, `sessions_thionville_20261014.csv`

Vérifiez qu'il y a bien 3 fichiers dans inbox. S'il en manque un, le pipeline attendra en vain puis s'arrêtera à l'étape de comptage : c'est voulu, c'est sa garde. Gardez aussi les 3 CSV sur votre ordinateur pour le test de l'étape 7.

2 Deux datasets génériques de plus

Auteur › **+** › **Jeu de données** › **Azure Data Lake Storage Gen2** › **Continuer**, puis le format. Service lié **LS_ADLS_Charge**, importer le schéma **Aucun**, chemin vide › **OK**. Puis, comme au guide n° 1 : trois paramètres **String** dans l'onglet **Paramètres**, et le chemin en contenu dynamique dans l'onglet **Connexion**.

format JSON
```
DS_ADLS_Json
```

format Binary
```
DS_ADLS_Binary
```

paramètres
```
container
```
```
folder
```
```
file
```

case conteneur
```
@dataset().container
```

case dossier
```
@dataset().folder
```

case fichier
```
@dataset().file
```

Pourquoi un dataset Binary ? Il ne lit pas le contenu, il déplace des octets. C'est l'outil des archives fidèles et des suppressions, quel que soit le format du fichier. Avec `DS_ADLS_Csv` et `DS_ADLS_Parquet`, vous avez désormais une boîte à outils complète, prête pour les guides suivants.

3 Le data flow : typer et contrôler les sessions

Auteur › **+** › **Flux de données**. Laissez le **Débogage du flux de données** éteint. Cliquez sur la toile vide › onglet **Paramètres** › **+ Nouveau**, type **double** :

nom du data flow
```
DF_Tarif_Audit
```

nom du paramètre
```
tolerance
```

valeur par défaut
```
0.5
```

Un paramètre de data flow sépare la règle du code. La tolérance vit dans le manifeste, le pipeline la transmet, le data flow l'applique : la changer ne demande aucune republication.

srcSessions

Source CSV

Ajouter une source · type **Jeu de données** · **DS_ADLS_Csv** · cochez **Autoriser la dérive de schéma**. N'importez aucune projection : les colonnes seront découvertes à l'exécution.

dcTyped

Colonne dérivée

Le **+** à droite de srcSessions › **Colonne dérivée**. Le CSV n'apporte que du texte : on crée quatre colonnes typées. Saisissez le nom, puis l'expression.

station_name
```
toString(byName('station'))
```

start_hour
```
toInteger(substring(toString(byName('start_time')), 12, 2))
```

energy
```
toDouble(byName('energy_kwh'))
```

price
```
toDouble(byName('price_eur'))
```

L'heure est lue directement dans le texte (caractères 12 et 13) : aucun fuseau horaire ne peut la décaler.

splitQuality

Fractionnement conditionnel

Première condition correspondante (First matching condition). Le premier flux reçoit les lignes inexploitables ; le flux par défaut, toutes les autres.

flux 1
```
rejected
```

flux par défaut
```
valid
```

condition de rejected
```
isNull(energy) || energy <= 0 || isNull(price) || isNull(start_hour)
```

4 Le data flow : appliquer la grille et classer les écarts

srcTariffs

Source JSON

Sous la première source, **Ajouter une source** · **DS_ADLS_Json** · dérive de schéma cochée · onglet **Options de la source** › **Paramètres JSON** › **Forme du document** : **Tableau de documents** (Array of documents), car le fichier contient une liste d'objets.

dcTariff

Colonne dérivée

Derrière srcTariffs, six colonnes typées :

tariff_station
```
toString(byName('station'))
```

peak_start
```
toInteger(byName('peak_start'))
```

peak_eur
```
toDouble(byName('peak_eur_kwh'))
```

peak_end
```
toInteger(byName('peak_end'))
```

offpeak_eur
```
toDouble(byName('offpeak_eur_kwh'))
```

session_fee
```
toDouble(byName('session_fee'))
```

selTariff

Sélectionner

Derrière dcTariff › **Sélectionner**. Décochez **Mappage automatique** : seules les six colonnes typées restent, gardez-les.

Pourquoi : les deux sources ont une colonne `station` ; sans ce tri, la jointure créerait deux colonnes de même nom et l'écriture échouerait.

lkpTariff

Recherche (Lookup)

Sur le flux **valid** › **Recherche**. Flux de recherche **selTariff** · **Faire correspondre plusieurs lignes** décoché · **N'importe quelle ligne** · condition **station_name** `==` **tariff_station**.

La Recherche garde toutes les sessions et n'ajoute qu'une ligne de grille à chacune : aucune session ne peut être dupliquée.

dcAudit

Colonne dérivée

Le cœur de l'audit. Prix juste = énergie × (tarif plein ou creux selon l'heure de début) + frais de session. Écart = prix payé − prix juste.

tariff_period
```
iif(start_hour >= peak_start && start_hour < peak_end, 'PLEINE', 'CREUSE')
```

expected_eur
```
round(energy * iif(start_hour >= peak_start && start_hour < peak_end, peak_eur, offpeak_eur) + session_fee, 2)
```

gap_eur
```
round(price - energy * iif(start_hour >= peak_start && start_hour < peak_end, peak_eur, offpeak_eur) - session_fee, 2)
```

verdict
```
iif(round(price - energy * iif(start_hour >= peak_start && start_hour < peak_end, peak_eur, offpeak_eur) - session_fee, 2) > $tolerance, 'SURFACTURE', iif(round(price - energy * iif(start_hour >= peak_start && start_hour < peak_end, peak_eur, offpeak_eur) - session_fee, 2) + $tolerance < 0, 'SOUS-FACTURE', 'CONFORME'))
```

winRank

Fenêtre (Window)

Derrière dcAudit › **Fenêtre**. Onglet **Over** : **station_name** · **Trier** : **gap_eur**, **Décroissant** · **Plage par** : par défaut · **Colonnes de fenêtre** :

nom
```
gap_rank
```

expression
```
rank()
```

Une fenêtre ne regroupe pas, elle compare : chaque session garde sa ligne et reçoit son rang dans sa station.

aggStation

Nouvelle branche + Agréger

Le **+** de winRank › **Nouvelle branche**, puis le **+** de la branche › **Agréger**. **Regrouper par** : **station_name**. Six agrégats :

sessions
```
count()
```

sous_factures
```
countIf(verdict == 'SOUS-FACTURE')
```

conformes
```
countIf(verdict == 'CONFORME')
```

overbilled_eur
```
round(sum(toDouble(iif(verdict == 'SURFACTURE', toString(gap_eur), '0'))), 2)
```

surfactures
```
countIf(verdict == 'SURFACTURE')
```

conformity_pct
```
round(toDouble(countIf(verdict == 'CONFORME')) * 100 / toDouble(count()), 1)
```

Les deux branches d'un `iif` doivent avoir exactement le même type : on les passe en texte, puis on reconvertit le tout en nombre.

snkAudit

Récepteur

Derrière **winRank**.

```
audit_sessions.csv
```

snkStation

Récepteur

Derrière **aggStation**.

```
station_audit.csv
```

snkRejects

Récepteur

Sur le flux **rejected**.

```
rejected_sessions.csv
```

Pour les trois : **DS_ADLS_Csv** · **Sortie dans un seul fichier** (Output to single file) · nom de fichier ci-dessus · **Effacer le dossier** · onglet **Optimiser** : **Partition unique** (Single partition), obligatoire pour un fichier unique.

Contrôle. Valider : aucune erreur, trois récepteurs. Gardez exactement ces noms : le pipeline les relit.

5 Le pipeline : attendre, vérifier, auditer

Auteur › **+** › **Pipeline**. Toile vide › **Variables** › quatre variables **String**, avec `0` comme valeur par défaut de `fileCount`. Reliez chaque activité à la suivante par la flèche verte **En cas de réussite**.

pipeline	variable 1	variable 2	variable 3	variable 4
`PL_Tarif_Audit`	`fileCount`	`rejectCount`	`qualityLevel`	`runSummary`

VAL_Manifest

Validation

Rien ne démarre tant que le manifeste n'existe pas. Jeu de données **DS_ADLS_Json**.

container	folder	file	Délai d'expiration	Mise en veille (s)	Taille minimale
`guide2`	`config`	`run_manifest.json`	`0.00:02:00`	`10`	`10`

Sans délai explicite, Validation attendrait 12 heures. Deux minutes suffisent pour un lot déjà déposé.

LKP_Manifest

Recherche (Lookup)

Lit les règles du lot. **DS_ADLS_Json**, mêmes paramètres, **Première ligne uniquement** coché : les règles se liront ensuite avec `output.firstRow`.

container	folder	file
`guide2`	`config`	`run_manifest.json`

UNT_WaitFiles

Jusque (Until)

Recompte les fichiers tant que le nombre attendu n'est pas atteint. Onglet **Paramètres** :

Expression

```
@greaterOrEquals(int(variables('fileCount')),
activity('LKP_Manifest').output.firstRow.expected_files)
```

Délai d'expiration : `0.00:05:00`

Onglet **Activités** › crayon. Trois activités reliées en vert :

GM_Inbox · Obtenir des métadonnées · **DS_ADLS_Csv** · champ **Éléments enfants**

- container : `guide2`
- folder : `inbox`
- file : `@string('')`

SV_FileCount · Définir une variable · **fileCount**

- Valeur : `@string(length(activity('GM_Inbox').output.childItems))`

WAIT_Poll · Attendre

- Temps d'attente (s) : `5`

> **Pourquoi attendre activement ?** Les fichiers arrivent rarement tous ensemble. La boucle compte, patiente cinq secondes, recompte, et n'avance que lorsque le lot est complet. Si un fichier manque, elle s'arrête au bout de cinq minutes au lieu de traiter un lot incomplet.

DF_RunAudit

Flux de données

Relié à UNT_WaitFiles. **Paramètres** : flux **DF_Tarif_Audit**, taille **Petit**. Paramètres des datasets :

Flux	container	folder	file
srcSessions	`guide2`	`inbox`	`@string('')`
srcTariffs	`guide2`	`reference`	`tariffs.json`
snkAudit	`guide2`	`gold/audit`	`@string('')`
snkStation	`guide2`	`gold/stations`	`@string('')`
snkRejects	`guide2`	`rejects`	`@string('')`

tolerance (Expression de pipeline) : `@activity('LKP_Manifest').output.firstRow.audit_tolerance_eur`

6 Le pipeline : la porte qualité, l'archive et le ménage

SV_Rejects

Définir une variable

Variable **rejectCount**. Compte les rejets, même s'il n'y en a aucun : dans ce cas le récepteur disparaît des métriques, d'où le `contains`.

```
@string(if(contains(activity('DF_RunAudit').output.runStatus.metrics, 'snkRejects'),
activity('DF_RunAudit').output.runStatus.metrics.snkRejects.rowsWritten, 0))
```

SV_Level

Définir une variable

Variable **qualityLevel**. Rejets au-delà du seuil du manifeste : CRITICAL. Quelques rejets : WARN. Aucun : OK. Le calcul se fait en entiers, sans division, donc sans arrondi trompeur.

```
@if(greater(mul(int(variables('rejectCount')), 100),
mul(add(activity('DF_RunAudit').output.runStatus.metrics.snkAudit.rowsWritten, int(variables('rejectCount'))),
activity('LKP_Manifest').output.firstRow.max_reject_pct)), 'CRITICAL', if(greater(int(variables('rejectCount')), 0),
'WARN', 'OK'))
```

SW_Quality

Basculer (Switch)

La porte qualité. Expression ci-dessous, puis **+ Ajouter un cas** deux fois (`OK` et `WARN`). Dans chaque cas, crayon › une activité.

expression

```
@variables('qualityLevel')
```

cas 1

```
OK
```

cas 2

```
WARN
```

Cas OK › Définir une variable **SV_SummaryOK**, variable **runSummary** :

```
@concat('Lot ', activity('LKP_Manifest').output.firstRow.batch_id, ' propre : ',
string(activity('DF_RunAudit').output.runStatus.metrics.snkAudit.rowsWritten), ' sessions auditées, ',
string(activity('DF_RunAudit').output.runStatus.metrics.snkStation.rowsWritten), ' stations au rapport')
```

Cas WARN › Définir une variable **SV_SummaryWarn**, variable **runSummary** :

```
@concat('Lot ', activity('LKP_Manifest').output.firstRow.batch_id, ' accepté avec réserves : ',
string(activity('DF_RunAudit').output.runStatus.metrics.snkAudit.rowsWritten), ' sessions auditées, ',
variables('rejectCount'), ' rejetées, ',
string(activity('DF_RunAudit').output.runStatus.metrics.snkStation.rowsWritten), ' stations au rapport')
```

Par défaut › Échec **FAIL_QualityGate** : le message, puis le code d'erreur.

```
@concat('Porte qualité fermée pour le lot ', activity('LKP_Manifest').output.firstRow.batch_id, ' : ',
variables('rejectCount'), ' lignes rejetées, au-delà du seuil de ',
string(activity('LKP_Manifest').output.firstRow.max_reject_pct), ' %')
```

```
QUALITY_GATE
```

> **Pourquoi un Switch plutôt qu'un If ?** La qualité n'est pas binaire. Trois niveaux, trois comportements, et un cas par défaut qui attrape tout ce qui n'est pas explicitement accepté : c'est la définition d'une porte sûre.

CP_Archive

Copier les données · binaire

Relié à SW_Quality. Copie tout le dossier, octet pour octet.

Source : **DS_ADLS_Binary**.

container

```
guide2
```

folder

```
inbox
```

file

```
@string('')
```

Récepteur : **DS_ADLS_Binary**, comportement de copie **Conserver la hiérarchie**.

container

```
guide2
```

folder

```
@concat('archive/',
activity('LKP_Manifest').output.firstRow.batch_id)
```

file

```
@string('')
```

DEL_Inbox

Supprimer

Relié à CP_Archive. Vide la boîte de réception.

Source : **DS_ADLS_Binary** · **Récursivement** décoché · onglet **Paramètres de journalisation** : **Activer la journalisation** décoché.

container

```
guide2
```

folder

```
inbox
```

file

```
@string('')
```

> **L'ordre est la sécurité.** Supprimer est définitif. On ne supprime donc qu'après une archive réussie, et seulement si la porte s'est ouverte.

7 Publier, exécuter, vérifier, puis éprouver la porte

1. **Tout valider** (Validate all), puis **Tout publier** (Publish all) › **Publier**.
2. **PL_Tarif_Audit** › **Ajouter un déclencheur** › **Déclencher maintenant** › **OK**, puis **Surveillance** › **Exécutions de pipeline**. Le data flow démarre d'abord son cluster Spark : c'est l'étape la plus longue.
3. Ouvrez le run, survolez **SV_SummaryWarn** › icône **Sortie**. Le champ `value` doit être :

```
Lot B-2026-10-14 accepté avec réserves : 57 sessions auditées, 3 rejetées, 3 stations au rapport
```

stchargeXXXX › **Navigateur de stockage** › **guide2** › **gold** › **stations** › **station_audit.csv** › **Modifier** :

station_name	sessions	conformes	surfactures	sous_factures	overbilled_eur	conformity_pct
Metz Gare	19	16	2	1	4.2	84.2
Nancy Stanislas	19	16	3	0	6.85	84.2
Thionville Centre	19	17	0	2	0.0	89.5

Total : 11,05 € de surfacturation. Dans `gold/audit/audit_sessions.csv` (57 lignes), le rang 1 de Metz est `MET-2004` (payé 19.99 €, prix juste 17.59 €, écart 2.4) et celui de Nancy `NAN-2011` (écart 3.1). `rejects` contient `MET-2900`, `NAN-2900` et `THI-2900`. `archive/B-2026-10-14/` contient les 3 CSV intacts, et `inbox` est vide.

Le test qui prouve tout : fermer la porte qualité

1. Redéposez les 3 CSV dans `guide2/inbox` (le dossier se recrée s'il a disparu).
2. **config** › **run_manifest.json** › **Modifier** : remplacez la ligne du seuil par celle-ci, puis **Enregistrer**.

```
  "max_reject_pct": 2,
```

Relancez. Le pipeline échoue sur **FAIL_QualityGate**, code `QUALITY_GATE` :

```
Porte qualité fermée pour le lot B-2026-10-14 : 3 lignes rejetées, au-delà du seuil de 2 %
```

Ni archive ni suppression : les 3 CSV sont toujours dans `inbox`. Remettez ensuite le seuil à `10`. Le pipeline a refusé un lot pour une raison métier, l'a expliqué en une phrase, et n'a touché à rien.

Si quelque chose bloque

Symptôme	Remède
UNT_WaitFiles expire	Moins de 3 fichiers dans `inbox` : ajoutez celui qui manque.
VAL_Manifest expire	Le manifeste n'est pas dans `guide2/config`.
« Incompatibilité de type »	Les branches d'un `iif` n'ont pas le même type : convertissez-les.
Colonnes en double à l'écriture	**Mappage automatique** resté coché dans selTariff.
Erreur sur `firstRow`	**Première ligne uniquement** décoché dans LKP_Manifest.
« Aucune valeur fournie pour le paramètre »	Saisissez `@string('')`.
GM_Inbox : dossier introuvable	Redéposez les CSV dans `inbox`.

8 Tout fermer

Ce pipeline n'a pas de déclencheur : il ne tourne que quand vous le lancez. Chaque exécution coûte surtout son data flow, quelques centimes.

Pour garder vos guides en sommeil

1. Vérifiez que le **Débogage du flux de données** est éteint : allumé, son cluster est facturé tant qu'il tourne.
2. Un pipeline inactif tout un mois peut être facturé quelques dizaines de centimes : supprimez ceux dont vous n'avez plus besoin.

Pour effacer seulement ce guide : supprimez le conteneur `guide2`, puis, dans ADF Studio, **PL_Tarif_Audit** et **DF_Tarif_Audit**. Le guide n° 1 reste intact.

Pour tout effacer : supprimez le groupe de ressources ci-dessous. Les guides n° 1 et n° 2 disparaissent ensemble.

```
rg-adf-charge
```